Lecture 17: IPv4, IPv6, NAT, and ICMP

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Identify and explain the purpose of each field in the IPv4 datagram header
2. Explain how ICMP provides error reporting and diagnostics for IP, and describe how `ping` and `traceroute` use ICMP at the protocol level
3. Describe IP fragmentation and reassembly, explain why it is problematic, and describe Path MTU Discovery as the modern alternative
4. Explain what NAT does, how port-address translation works, and why NAT complicates the end-to-end principle
5. Compare the IPv4 and IPv6 headers, identifying what was added, removed, and simplified
6. Describe the current state of IPv6 deployment and explain why the transition has taken decades
7. Outline the DHCP process (Discover/Offer/Request/Ack) and explain how hosts obtain IP addresses dynamically

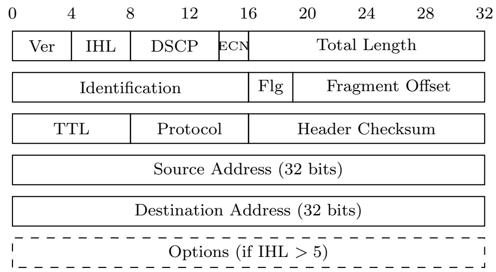
1 From Routing to the Datagram

In Lectures 13–16, we built up the complete routing picture: forwarding tables and longest-prefix match (L13), IP addresses and subnets (L14), Dijkstra and link-state routing inside an AS (L15), and Bellman-Ford, distance-vector, and BGP between ASes (L16). We know *how* a router decides where to send a packet.

This lecture opens up the **IPv4 datagram** itself—the actual data structure that every router examines—and covers the protocols that support it: ICMP for error reporting, NAT for address sharing, IPv6 as the next-generation replacement, and DHCP for dynamic address configuration.

2 The IPv4 Datagram Header

Every IPv4 packet begins with a header of at least 20 bytes (160 bits). The header contains all the information routers need to forward the packet.



2.1 Field-by-Field Description

Version (4 bits). Always 4 for IPv4. Tells the router which header format to expect (IPv6 = 6).

IHL—Internet Header Length (4 bits). Header length in 32-bit words. Minimum is 5 (= 20 bytes, no options); maximum is 15 (= 60 bytes). Most packets have IHL = 5.

DSCP—Differentiated Services Code Point (6 bits). IP's practical QoS mechanism. Packets are marked by traffic class so that routers can give preferential treatment:

DSCP and QoS

DSCP provides differentiated service *without* per-flow signaling. The sender (or edge router) marks packets; each router independently decides how to treat each class—prioritize in queues, drop lower-priority traffic first during congestion. This completes the QoS story introduced in L13: the internet is best-effort by default, but DSCP lets operators differentiate when they choose to.

DSCP Value	Per-Hop Behavior	Typical Use
000000	Best Effort (default)	Regular web traffic
001010–011010	Assured Forwarding	Video streaming
101110	Expedited Forwarding	VoIP—low delay, low jitter

ECN—Explicit Congestion Notification (2 bits). Allows routers to signal congestion *without* dropping packets. The sender marks the packet as "ECN-capable," and a congested router can set the "Congestion Experienced" bit instead of dropping the packet. We will revisit ECN when we study TCP congestion control.[1]

Total Length (16 bits). Total datagram size in bytes (header + payload). Maximum is 65,535 bytes, but in practice datagrams are limited by the link's MTU (typically 1,500 bytes for Ethernet).

Identification (16 bits). Uniquely identifies a datagram. All fragments of the same original datagram share the same Identification value, enabling reassembly at the destination.

Flags (3 bits).

- Bit 0: Reserved (must be 0).
- Bit 1: **DF (Don't Fragment)**—if set, the datagram must not be fragmented. If a router cannot forward it without fragmenting, it drops the packet and sends an ICMP error.
- Bit 2: **MF (More Fragments)**—set on all fragments except the last one.

[1] Wikipedia: Explicit Congestion Notification—ECN bits in IPv4/IPv6, interaction with TCP.

Fragment Offset (13 bits). Position of this fragment's data within the original datagram, measured in 8-byte units. The first fragment has offset 0.

TTL—Time to Live (8 bits). Each router decrements TTL by 1. When TTL reaches 0, the router drops the packet and sends an ICMP Time Exceeded message. This prevents packets from looping forever through routing loops—we saw this mechanism in action in L13 with `traceroute`. Typical initial values: 64 (Linux), 128 (Windows), 255 (network equipment).

Protocol (8 bits). Identifies the upper-layer protocol carried in the payload—the **demultiplexing key** that tells the destination host's network layer which transport-layer module should receive the data.

Value	Protocol
1	ICMP
6	TCP
17	UDP
89	OSPF

This is analogous to the EtherType field in Ethernet frames (L08).

Header Checksum (16 bits). Covers the header only (not the payload—upper layers have their own checksums). Recomputed at every hop because TTL changes. This per-hop recomputation is one reason IPv6 removed the header checksum entirely.

Source Address (32 bits). The sender's IP address. Set by the originating host and normally not modified in transit. Exception: NAT, covered in Section 5.

Destination Address (32 bits). The recipient's IP address. Every router uses this for the forwarding table lookup (longest prefix match, from L14).

Options (0–40 bytes, variable). Rarely used today. Historically included Record Route, Timestamp, and Source Route. Options complicate router fast-path processing because the header length varies—another reason IPv6 moved optional features to extension headers.

2.2 What Routers Actually Examine

Not every field matters at every hop. A forwarding router primarily needs:

- **Destination Address**—for the forwarding decision (longest prefix match)
- **TTL**—decrement and check
- **Header Checksum**—verify and recompute after TTL change
- **Total Length**—to know where the packet ends
- **Flags / Fragment Offset**—only if fragmentation is needed

The Source Address, Protocol, DSCP, and payload generally pass through unmodified (except by NAT or QoS-aware routers).[2]

[2]Wikipedia: IPv4 header—field-by-field description of the IPv4 datagram header.

3 ICMP: The Internet Control Message Protocol

The **Internet Control Message Protocol (ICMP)** is the network layer's mechanism for error reporting and diagnostics. Defined in RFC 792, ICMP is technically a separate protocol from IP but is encapsulated directly in IP datagrams (Protocol field = 1) and is considered part of the network layer.[3]

ICMP exists because IP is unreliable and connectionless. When something goes wrong—a packet is dropped, a destination is unreachable, or a TTL expires—the network needs a way to report the problem back to the sender. That mechanism is ICMP.

3.1 ICMP Message Format

Every ICMP message begins with a Type (8 bits), Code (8 bits), and Checksum (16 bits), followed by type-specific data. For error messages, the body includes the IP header and first 8 bytes of the original datagram that triggered the error—enough for the sender to identify which packet failed (the 8 bytes cover the transport-layer source and destination ports).

Key ICMP Message Types

Type	Code	Name	Purpose
0	0	Echo Reply	Response to ping
3	0	Dest Unreachable: Network	No route to destination network
3	1	Dest Unreachable: Host	Network reachable but host unreachable
3	3	Dest Unreachable: Port	Host reached but destination port closed
3	4	Frag Needed, DF Set	Packet too large, DF flag prevents fragmentation
8	0	Echo Request	Ping request
11	0	Time Exceeded: TTL	TTL decremented to 0 (used by traceroute)
11	1	Time Exceeded: Reassembly	Fragment reassembly timeout

Important rule: ICMP error messages are never generated in response to other ICMP error messages. This prevents infinite cascades of error reports.

3.2 How `ping` Works

In L13, we used `ping` as a diagnostic tool without examining the underlying protocol. Now we can see the full mechanism:

1. The host creates an **ICMP Echo Request** (Type 8, Code 0) containing an identifier, sequence number, and optional payload data.
2. This ICMP message is encapsulated in an IP datagram with Protocol = 1.
3. The datagram is routed to the destination through the forwarding and routing machinery from L13–L16.

[3] Wikipedia: Internet Control Message Protocol—ICMP message types, ping, traceroute implementation.

4. The destination host's network layer sees Protocol = 1 and hands the payload to ICMP.
5. ICMP generates an **ICMP Echo Reply** (Type 0, Code 0) with the same identifier, sequence number, and payload.
6. The reply is routed back to the original sender.
7. The sender measures the round-trip time (RTT).

`ping` does not use TCP or UDP—it operates directly at the network layer using ICMP.

3.3 How `traceroute` Works

`traceroute` exploits TTL expiration and ICMP Time Exceeded messages to discover each router along a path:

1. Send a packet with **TTL = 1**. The first router decrements TTL to 0, drops the packet, and sends an **ICMP Time Exceeded** (Type 11, Code 0) reply. The source IP of this reply reveals the first router's address.
2. Send a packet with **TTL = 2**. The second router replies with Time Exceeded.
3. Repeat with TTL = 3, 4, 5, ... until the destination is reached.
4. The destination responds differently depending on the implementation:
   - **Unix `traceroute`**: sends UDP packets to a high port number (33434+). The destination replies with **ICMP Destination Unreachable: Port** (Type 3, Code 3) because nothing is listening on that port.
   - **Windows `tracert`**: sends ICMP Echo Requests. The destination replies with an Echo Reply.

The `* * *` entries in `traceroute` output indicate routers that are configured not to send ICMP replies (rate-limited or disabled for security reasons).

4 IP Fragmentation

4.1 The MTU Problem

Every link in the network has a **Maximum Transmission Unit (MTU)**—the largest frame payload the link can carry:

Link Type	MTU
Ethernet	1,500 bytes
PPPoE (DSL)	1,492 bytes
WiFi (802.11)	2,304 bytes
Jumbo frames	9,000 bytes

When a router needs to forward an IP datagram onto a link whose MTU is smaller than the datagram, it must either **fragment** the datagram into smaller pieces or **drop** it (if the DF flag is set) and send an ICMP error.[4]

[4]Wikipedia: IP fragmentation— fragmentation and reassembly, MTU, fragment offset calculation.

4.2 How Fragmentation Works

The router splits the datagram's payload into fragments, each small enough for the outgoing link's MTU. Each fragment gets its own IP header with the same Identification value, the appropriate MF flag, and the correct Fragment Offset.

Fragmentation Example

A 4,000-byte datagram (20-byte header + 3,980 bytes payload) enters a link with MTU = 1,500:

Fragment	Total Length	MF	Fragment Offset	Payload Bytes
1	1,500	1	0	0–1,479
2	1,500	1	185 (= 1480/8)	1,480–2,959
3	1,040	0	370 (= 2960/8)	2,960–3,979

Reassembly happens only at the **destination**—not at intermediate routers. The destination collects all fragments with the same Identification, uses Fragment Offset to order them, and waits for the fragment with MF = 0 to determine the total size.

4.3 Why Fragmentation Is Problematic

1. **Losing one fragment loses the entire datagram**: if fragment 2 of 3 is lost, the destination cannot reassemble the original. The other two fragments consumed bandwidth for nothing.
2. **Reassembly is complex**: the destination must buffer out-of-order fragments, handle duplicates, and time out incomplete reassemblies.
3. **Security vulnerabilities**: fragmentation has been exploited in attacks—the "Ping of Death" used overlapping fragments to crash reassembly code, and "teardrop" attacks sent contradictory fragment offsets.
4. **Header overhead**: each fragment carries its own 20-byte IP header, increasing the total data transmitted.

4.4 Path MTU Discovery

The modern approach avoids fragmentation entirely:[5]

1. The sender sets the **DF (Don't Fragment)** flag on all outgoing datagrams.
2. If a router encounters an MTU too small for the packet, it drops the packet and sends **ICMP Destination Unreachable: Fragmentation Needed** (Type 3, Code 4), including the MTU of the problematic link.
3. The sender reduces its datagram size to fit that MTU and retransmits.
4. This repeats until the sender discovers the smallest MTU along the entire path—the **path MTU**.

[5] Wikipedia: Path MTU Discovery— PMTUD mechanism, black holes, RFC 1191.

PMTUD is used by TCP, which adjusts its Maximum Segment Size (MSS) accordingly.

PMTUD Black Holes

Some firewalls block all ICMP traffic, preventing the "Fragmentation Needed" reply from reaching the sender. This creates a **PMTUD black hole**: the sender keeps sending large packets that get silently dropped, and the connection stalls. This is why network operators should *never* indiscriminately block all ICMP.

Demo: Fragmentation and DF Flag

```
# Send a large ping that exceeds Ethernet MTU (1500 bytes)
$ ping -s 3000 -c 1 google.com
# => fragmented into multiple IP datagrams

# Try with Don't Fragment set
$ ping -D -s 2000 -c 1 google.com
# => "Message too long" (ICMP Fragmentation Needed)

# In Wireshark: filter on 'icmp'
# Type 8 (Echo Request) going out, Type 0 (Echo Reply) coming back
# IP header shows Protocol = 1
```

5 NAT: Network Address Translation

5.1 The Problem NAT Solves

IPv4 has only $2^{32} \approx 4.3$ billion addresses, and the pool is exhausted. Yet there are far more than 4.3 billion devices connected to the internet. **NAT (Network Address Translation)** resolves this by allowing an entire private network to share a single public IP address.[6]

Combined with RFC 1918 private address ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) from L14, NAT is what kept the internet running long past the point where IPv4 addresses should have been exhausted.

5.2 How NAT Works

A NAT-enabled router sits between the private network and the public internet. When a host on the private network sends a packet:

1. **Outgoing**: The NAT router replaces the source IP (e.g., 10.0.0.2) with its public IP (e.g., 128.197.10.1) and replaces the source port with a unique port from its pool (e.g., 40001). It records this mapping in the **NAT translation table**.

2. **Return**: When the reply arrives addressed to 128.197.10.1:40001, the router looks up port 40001 in the table, finds the mapping to 10.0.0.2:5001, and forwards the packet with the original destination address and port restored.

[6] Wikipedia: Network address translation—NAT types, port translation, implications for end-to-end principle.

NAT Translation Table

WAN Side	LAN Side
128.197.10.1:40001	10.0.0.2:5001
128.197.10.1:40002	10.0.0.3:5002
128.197.10.1:40003	10.0.0.4:5003

This is **Port Address Translation (PAT)** or NAPT—the most common form. A single public IP can support up to ~65,000 simultaneous connections (limited by the 16-bit port space).

NAT modifies both the IP header (source/destination address) *and* the transport header (port numbers), and must recompute both the IP header checksum and the TCP/UDP checksum.

5.3 NAT and the End-to-End Principle

NAT fundamentally violates the **end-to-end principle**—the design philosophy that network functions should be implemented at the endpoints, not inside the network:

What NAT Breaks

- **Incoming connections are broken**: A host behind NAT cannot be directly reached from the public internet. Running a server on your home network requires port forwarding or similar workarounds.
- **Peer-to-peer is complicated**: Two hosts both behind NAT cannot directly communicate. Each sees only the other's public NAT address, and neither can initiate a connection.
- **Protocol assumptions violated**: IP was designed so that every host has a globally unique address. Applications that embed IP addresses in their payload (FTP, SIP) need special NAT handling (Application Layer Gateways).

Workarounds include port forwarding (manual), UPnP/NAT-PMP (automatic), STUN/TURN/ICE (used by WebRTC and VoIP), and UDP hole punching (used by gaming and P2P applications).

5.4 NAT's Legacy

NAT is simultaneously the hack that saved the internet from IPv4 exhaustion, a violation of clean network design, a de facto security feature (hosts behind NAT are not directly reachable), and the primary reason IPv6 adoption stalled—NAT made IPv4 "good enough" for another 20+ years.

6 IPv6: The Next Generation

6.1 The Original Vision

IPv6 was designed in the mid-1990s (RFC 2460, 1998) to solve IPv4 address exhaustion:[7]

[7] Wikipedia: IPv6—address format, header structure, deployment status.

- **Vast address space**: 128-bit addresses providing $2^{128} \approx 3.4 \times 10^{38}$ addresses
- **Global addressability**: every device gets a real, globally routable address—no NAT needed
- **Simplified header**: fixed 40 bytes, no header checksum, faster router processing
- **No fragmentation in transit**: only endpoints may fragment
- **Auto-configuration (SLAAC)**: hosts can configure themselves without DHCP
- **Mandatory IPsec**: built-in encryption and authentication

6.2 The Reality Check

The IPv4-to-IPv6 transition has been the longest upgrade in internet history—over 25 years and counting.[8]

NAT delayed the crisis: NAT unexpectedly extended IPv4's life by allowing millions of devices to share a single public address. The perceived urgency of IPv6 evaporated.

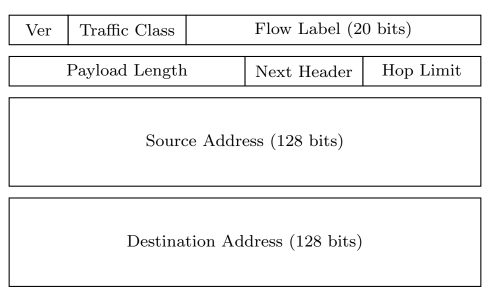
Enterprise adoption stalled: For most companies, IPv4 + NAT works fine. The cost of upgrading (dual-stack infrastructure, staff training, application testing) far exceeds the perceived benefit.

Mobile carriers drove actual deployment: Mobile carriers ran out of IPv4 addresses first because of the sheer number of smartphones. T-Mobile US runs over 90% IPv6 traffic. Reliance Jio (India) launched as an IPv6-first network. Your phone likely has a real global IPv6 address—no NAT needed.

Current state: approximately 45% of Google traffic arrives over IPv6 (as of 2025), but the distribution is extremely uneven—some countries exceed 60% (India, France, Germany) while others are below 10%. IPv6 is largely a mobile-internet and consumer-broadband story, not the universal enterprise transition that was envisioned.

6.3 IPv6 Header Format

The IPv6 header is **fixed at 40 bytes**:



[8] Wikipedia: IPv6 deployment— adoption statistics by country and carrier, Google IPv6 measurements.

6.4 IPv4 vs. IPv6 Comparison

Feature	**IPv4**	**IPv6**
Address size	32 bits	128 bits
Header size	20–60 bytes (variable)	40 bytes (fixed)
Header checksum	Yes (recomputed every hop)	Removed
Fragmentation	In base header; routers fragment	Removed from base header
Options	In base header (variable IHL)	Extension headers
TTL	Time to Live	Hop Limit (renamed)
Protocol	Identifies upper layer	Next Header (same + chains extensions)
Broadcast	Yes	No—multicast only
Auto-config	DHCP required	SLAAC (stateless)

6.5 What Was Removed and Why

IPv6 Header Simplifications

1. **Header checksum removed**: Recomputing it at every hop wastes CPU. Link-layer CRCs catch bit errors per link; transport-layer checksums catch end-to-end corruption. The IP header checksum was redundant work.

2. **Fragmentation fields removed**: In IPv6, routers *never* fragment. If a packet is too large, the router sends ICMPv6 "Packet Too Big." Only the source host may fragment, using an extension header.

3. **Options / IHL removed**: Replaced by **extension headers**—a chain of optional headers between the fixed IPv6 header and the payload. The Next Header field in each header points to the next, forming a linked list.

6.6 IPv6 Address Notation

IPv6 addresses are written as eight groups of four hexadecimal digits separated by colons:[9]

`2001:0db8:85a3:0000:0000:8a2e:0370:7334`

Two compression rules reduce this to a shorter form:

Rule 1—Drop leading zeros in each group: `0db8` → `db8`, `0370` → `370`, `0000` → `0`:

`2001:db8:85a3:0:0:8a2e:370:7334`

Rule 2—Replace one longest run of consecutive all-zero groups with `::`:

`2001:db8:85a3::8a2e:370:7334`

The `::` Rule

`::` can appear **only once** per address. If it appeared twice (e.g., `2001::1::2`), there would be no way to determine how many zero groups each `::` replaces. To expand a compressed address, count the groups explicitly present and fill in zeros to make exactly 8 groups total.

[9] Wikipedia: IPv6 address—notation, address types, scopes, special addresses.

Expansion examples:

- `::1` = `0000:0000:0000:0000:0000:0000:0000:0001` (loopback—7 zero groups + `0001`)
- `fe80::1` = `fe80:0000:0000:0000:0000:0000:0000:0001` (link-local)
- `2001:db8::1:0:0:1`—5 groups present (`2001:db8`, `1`, `0`, `0`, `1`), so `::` expands to 3 zero groups: `2001:0db8:0000:0000:0000:0001:0000:0001`. You can verify: $5 + 3 = 8$ groups.

6.7 IPv6 Address Types

Type	Prefix	Scope	Purpose
Global Unicast	`2000::/3`	Internet	Globally routable
Link-Local	`fe80::/10`	Single link	Auto-configured; neighbor discovery
Unique Local	`fc00::/7`	Organization	Like RFC 1918 private addresses
Multicast	`ff00::/8`	Varies	Replaces broadcast
Loopback	`::1/128`	Host	Equivalent to 127.0.0.1

Every IPv6 interface has *at least two* addresses: a **link-local** address (auto-generated, always present) and typically a **global unicast** address. There is no broadcast in IPv6—all one-to-many communication uses **multicast** (`ff02::1` = all nodes, `ff02::2` = all routers).[10]

6.8 IPv6 Address Allocation Hierarchy

IPv6 address space is allocated in a structured hierarchy with a critical architectural boundary at /64:

From	To	Typical Prefix	What It Provides
RIR (e.g., ARIN)	ISP	**/32** (or /29 for large ISPs)	65,536 customer /48 blocks
ISP	Enterprise	**/48**	65,536 subnets (each a /64)
ISP	Home / small site	**/48** or **/56**	256–65,536 subnets
Customer	Single subnet	**/64**	$2^{64} \approx 1.8 \times 10^{19}$ host addresses

The /64 Boundary

The /64 subnet size is **mandatory** in IPv6. SLAAC (Stateless Address Auto-Configuration) and Neighbor Discovery Protocol both assume that the host portion of the address is exactly 64 bits. You cannot use /65 or /80 subnets and expect auto-configuration to work. This is a fixed architectural decision, not a convention.

Contrast with IPv4: In IPv4, subnets can be any size (/24, /26, /30, etc.) and address conservation is critical. In IPv6, a single /64 subnet has more addresses than the entire IPv4 address space *squared*. The design philosophy shifted from "allocate the minimum" to "allocate generously and keep the hierarchy clean." An organization with a /48 has 65,536 subnets available—more than enough for one per floor, per VLAN, or per department, without ever worrying about address exhaustion.

[10] Wikipedia: ICMPv6—IPv6 version of ICMP, Neighbor Discovery Protocol.

6.9 Transition Mechanisms

The internet cannot switch from IPv4 to IPv6 overnight. Three transition mechanisms coexist:

Dual stack: Hosts and routers run both IPv4 and IPv6 simultaneously. DNS returns both A (IPv4) and AAAA (IPv6) records, and the host chooses which to use. Modern systems prefer IPv6 when available ("Happy Eyeballs" algorithm, RFC 8305). This is the most common approach.

Tunneling (6in4, 6to4, Teredo): IPv6 packets are encapsulated inside IPv4 packets to cross IPv4-only segments. This allows IPv6 "islands" to communicate across an IPv4 "ocean."

NAT64 / DNS64: Translates between IPv6-only clients and IPv4-only servers. The translator maps IPv4 addresses into a special IPv6 prefix (`64:ff9b::/96`). Used by mobile carriers that run IPv6-only internally.

Demo: IPv6 on Your Machine

```
# See both IPv4 and IPv6 addresses
$ ifconfig en0
  inet 192.168.1.42         <- IPv4 (probably behind NAT)
  inet6 fe80::1a2b:3c4d     <- link-local (always present)
  inet6 2601:1234:abcd::42  <- global (if ISP supports IPv6)

# Ping over IPv6
$ ping6 google.com

# Compare: your phone likely has a real global IPv6 address
# from your mobile carrier -- no NAT needed!
```

7 DHCP: Dynamic Host Configuration

7.1 The Bootstrap Problem

When a host first connects to a network, it has no IP address, no subnet mask, no default gateway, and no DNS server. **DHCP—Dynamic Host Configuration Protocol** automates the configuration process: a DHCP server on the network hands out IP addresses and parameters to hosts that request them.[11]

[11] Wikipedia: Dynamic Host Configuration Protocol—DORA process, leases, relay agents.

7.2 The DHCP Four-Step Dance (DORA)

DHCP Message Exchange

1. **Discover**: The client broadcasts a request (destination 255.255.255.255, source 0.0.0.0) because it knows no server address and has no IP address of its own.
2. **Offer**: The server responds with an available IP address, subnet mask, default gateway, DNS server addresses, and a lease duration.
3. **Request**: The client accepts the offer. This message is also broadcast so that other DHCP servers (if any) know the address was claimed.
4. **Ack**: The server confirms the assignment. The client can now use the address for the duration of the lease.

7.3 Key Details

Leases: Addresses are assigned for a limited time (hours to days). The client must renew before the lease expires, or the address returns to the pool. This enables address reuse across devices that connect and disconnect.

Relay agents: If the DHCP server is on a different subnet, a relay agent (typically the router) forwards DHCP broadcasts as unicast to the server. Without relays, every subnet would need its own DHCP server.

Transport: DHCP uses UDP—port 67 (server) and port 68 (client). It cannot use TCP because the client has no IP address to establish a TCP connection.

7.4 IPv6: SLAAC and DHCPv6

IPv6 provides two mechanisms for address configuration:

SLAAC (Stateless Address Auto-Configuration): The host generates its own global address from the network prefix (advertised by the router via Router Advertisement messages) and a host identifier (derived from the MAC address or generated randomly for privacy). No DHCP server needed.[12]

DHCPv6: Works in "stateful" mode (assigns addresses) or "stateless" mode (provides only DNS servers and other options while SLAAC handles the address).

In practice, most IPv6 networks use SLAAC for address assignment and either SLAAC options or DHCPv6 for DNS configuration.

[12] Wikipedia: SLAAC—stateless address auto-configuration in IPv6.

8 Summary

Topic	Key Takeaway
IPv4 header	20-byte base with TTL (hop counter), Protocol (demux key), Source/Dest addresses; DSCP for QoS
ICMP	Error reporting (Destination Unreachable, Time Exceeded) and diagnostics (Echo Request/Reply)—the protocol behind `ping` and `traceroute`
Fragmentation	Router fragmentation is problematic; PMTUD avoids it; IPv6 bans in-transit fragmentation entirely
NAT	Saved IPv4 from address exhaustion but broke end-to-end communication; uses port translation
IPv6	128-bit addresses, simplified fixed header, no broadcast; real adoption driven by mobile carriers
DHCP	Discover/Offer/Request/Ack; IPv6 adds SLAAC for stateless auto-configuration

With L13–L17, the network layer is complete. L18 (Thursday) is a hands-on Wireshark lab tying together all the protocols from this unit: ICMP, IPv4/IPv6 headers, DHCP, and NAT.

References

- Wikipedia: IPv4—address format, header structure, exhaustion timeline
- Wikipedia: Internet Control Message Protocol—ICMP message types, ping, traceroute
- Wikipedia: IP fragmentation— fragmentation and reassembly, MTU
- Wikipedia: Path MTU Discovery—PMTUD mechanism, black holes
- Wikipedia: Network address translation—NAT types, port translation
- Wikipedia: IPv6—address format, header structure, deployment
- Wikipedia: IPv6 address—notation, types, scopes
- Wikipedia: IPv6 deployment— adoption statistics by country and carrier
- Wikipedia: DHCP—DORA process, leases, relay agents
- Wikipedia: Differentiated services—DSCP, per-hop behaviors
- Wikipedia: Explicit Congestion Notification—ECN bits, interaction with TCP
- Wikipedia: ICMPv6—Neighbor Discovery Protocol